



AccoTEST STS8300 应用培训

ACM 硬件及编程

2022-1-11

Make Testing More Valuable

目录

- 单板特性
- 指标
- 原理框图及外观结构
- 脉冲输出功率
- 槽位及对应PogoPin定义
- VI源功能
- AWG功能

• 单板特性

SM8226 模拟通道板是 STS 8300 系统中的标准模块，模块提供二十四路精密四象限VI源通道，每六个通道为一组。

- 单板 24 通道，每 6 通道为一组（每组共用一个地线补偿信号 DGS）
- 四象限输出电压电流
- 带远端地线补偿的 Kelvin 连接
- 1 个电压量程档：
N2P18V
- 7 个电流量程档：
 $\pm 500\text{mA}$ (脉冲), $\pm 200\text{mA}$, $\pm 20\text{mA}$, $\pm 2\text{mA}$, $\pm 200\mu\text{A}$, $\pm 20\mu\text{A}$, $\pm 5\mu\text{A}$
- 每通道具备硬件数据处理功能，包括最大值、最小值、平均值及跳变位置
- 每通道具备可编程窗口式电压及电流箝位
- 支持与 STS 8300 系统内模块同步功能
- 支持多工位并行测试工作模式
- 本板保存校准数据
- 本板自带温度监测功能

- 指标

Voltage Force

Range	Resolution(16-Bit)	Accuracy
-2V to +18V	0.305mV	$\pm(3\text{mV} + 0.025\% \text{ Rdg})$

Voltage Measure

Range	Resolution(16-Bit)	Accuracy
-2V to +18V	0.610mV	$\pm(3\text{mV} + 0.025\% \text{ Rdg})$

指标

Current Force

Range	Resolution(16-Bit)	Accuracy
±500mA	15.259uA	±(0.2mA + 0.2% Rdg)
±200mA	6.104uA	±(40uA + 0.05% Rdg)
±20mA	0.610uA	±(4uA + 0.05% Rdg)
±2mA	61.035nA	±(400nA + 0.05% Rdg)
±200uA	6.104nA	±(40nA + 0.05% Rdg)
±20uA	0.610nA	±(10nA + 0.05% Rdg)
±5uA	0.153nA	±(5nA + 0.05% Rdg)

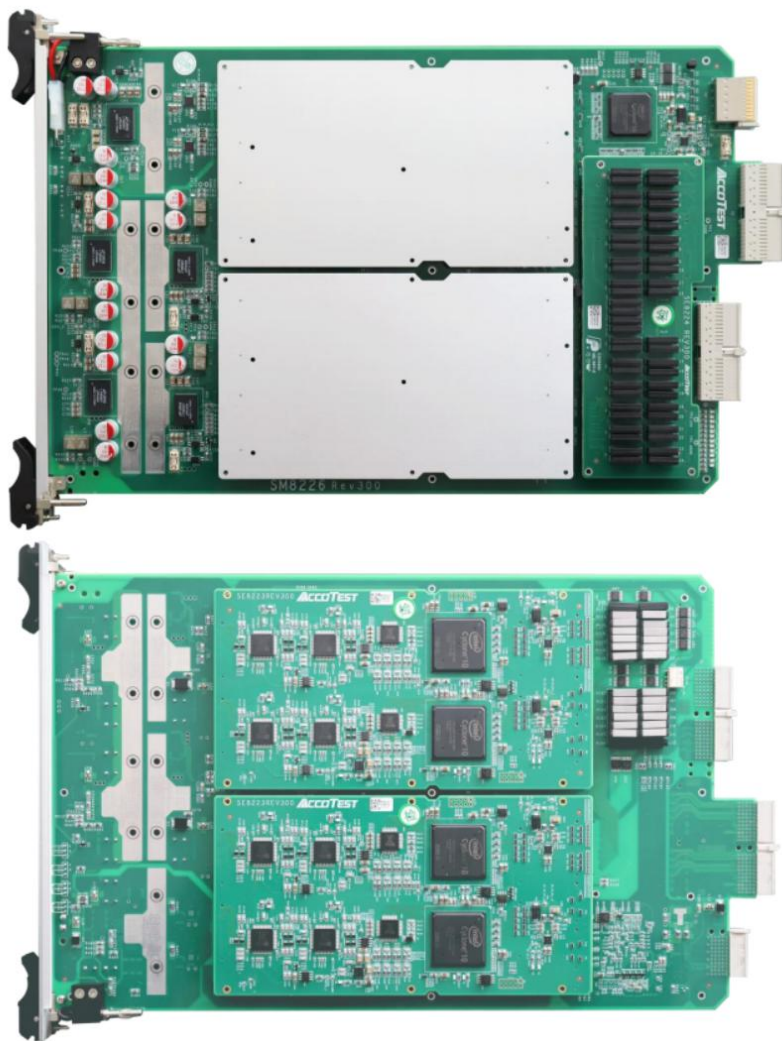
Current Measure

Range	Resolution(16-Bit)	Accuracy
±500mA	30.517uA	±(0.2mA + 0.2% Rdg)
±200mA	12.207uA	±(60uA + 0.05% Rdg)
±20mA	1.221uA	±(6uA + 0.05% Rdg)
±2mA	122.07nA	±(600nA + 0.05% Rdg)
±200uA	12.207nA	±(60nA + 0.05% Rdg)
±20uA	1.221nA	±(10nA + 0.05% Rdg)
±5uA	0.305nA	±(6nA + 0.05% Rdg)

- 指标

AWG	Resolution	Pattern Depth / Per Channel	DA Update Rate	AWG step
	16bit	4K	100KHz	100us
Digitizer	Resolution	Capture Memory / Per Channel	Maximum Sample Rate	
	16bit	4K	200kSPS	

- 外观结构



- **脉冲输出功率**

当选择 500mA 档位时，输出电流约大于 215mA 时会启动 FPGA 的定时器，定时 10ms 后会自动将电流档位从 500mA 切换到 20mA 档位。

槽位及对应PogoPin定义

ACM对应的Analog槽位（TIB类型相关）

33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	CBITE
CBITE	Analog	Analog	Analog	Analog	Analog	Analog	Analog	Analog	DCM	QTMUe	Analog	Analog	Analog	Analog	Analog	Analog	Cal
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35

A28D4 Config

33	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	34
NULL	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CBITE
CBITE	Analog	Analog	Analog	Analog	Analog	Analog	TMU	Digital	Digital	TMU	Analog	Analog	Analog	Analog	Analog	Analog	CAL
36	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	35

A24D8 Config

槽位及对应PogoPin定义

From PogoPin Tool

P111_slot12slot11

1	2	3	4	5	6	7	8	9	10	11	12
	FH_0	SH_0	G_0				FH_0	FH_0	G_0	AGND	
	FL_0	SL_0					FL_0	FL_0		AGND	
FH_0	FH_0	G_0	FL_0	FL_0		FH_1	SH_1	G_1	FL_1	SL_1	
FH_0	SH_0	FH_4	FH_6	SH_6	FH_10	FL_1	FL_1				
FH_1	SH_1	SH_4	FH_7	SH_7	SH_10						
FL_0-5	FL_0-5	SL_0-5	FL_6-11	FL_6-11	SL_6-11	FH_1	FH_1	G_1			
FH_2	SH_2	FH_5	FH_8	SH_8	FH_11	FH_1	FH_1	G_1			
FH_3	SH_3	SH_5	FH_9	SH_9	SH_11	FL_1	FL_1				
FH_12	SH_12	FH_16	FH_13	SH_13	SH_16	FH_14	SH_14	FH_17	FH_15	SH_15	SH_17
	FH_18	SH_18	FH_22	FL_12-17	FL_12-17	SL_12-17	FH_20	SH_20	FH_23	AGND	
	FH_19	SH_19	SH_22	FL_18-23	FL_18-23	SL_18-23	FH_21	SH_21	SH_23	AGND	

Open Config slot

Resource:
ACM

Slot:
slot12

Serial NO:
0

☒ AutoSwitch

Config File Path:
C:/AccoTest/AccoTEST System/
ConfigEx. slot

Cell Channel And Slot:

P000 P001 P002 P003 P004 P005 P006 P007 P008 P009

P010 P011 P012 P013 P014 P015 P016 P017 P018 P019

P020 P021 P022 P023 P024 P025 P026 P027 P028 P029

P030 P031 P032 P033 P034 P035 P036 P037 P038 P039

- VI源功能

Set ();

SetSyn ();

SetClamp ();

MeasureVI ();

GetMeasResult ();

BlockRead ();

Pulse ();

• ACM

Set (VIMode *viMode*, double *setValue*, ACM_VRNG *vRange*, ACM_IRNG *iRange*, ACM_OUT_RELAY *relayStatus* = ACM_RELAY_HOLD, double *risingTime* = 0.1);

Remarks

设置ACM的状态，包括工作模式、恒压或恒流值、电压量程或电流量程以及输出继电器的状态。

Parameters

viMode ACM 工作模式，有两种模式：

FV: 选择恒压模式

FI: 选择恒流模式

setValue double 型变量，设置恒压或者恒流值。

选择恒压模式时，该参数为恒压值，单位为 V。最大范围为-2 ~ +18。

选择恒流模式时，该参数为恒流值，单位为 A。最大范围为-0.5 ~ +0.5。

vRange ACM 电压量程：

ACM_N2P18V

iRange ACM 电流量程：

ACM_500MA, ACM_200MA, ACM_20MA,

ACM_2MA, ACM_200UA, ACM_20UA, ACM_5UA

ACM_2MA, ACM_200UA, ACM_20UA and ACM_5UA, 称为内部电流量程；

ACM_500MA, ACM_200MA and ACM_20MA , 称为外部电流量程。

当电流挡位在内部量程和外部量程之间切换时，输出继电器会先断开，然后再闭合。

• ACM

Set (VIMode *viMode*, double *setValue*, ACM_VRNG *vRange*, ACM_IRNG *iRange*, ACM_OUT_RELAY *relayStatus* = ACM_RELAY_HOLD, double *risingTime* = 0.1);

注意:

- (1) 选择 ACM_500mA 档位时，当电流绝对值小于 215mA 时，板卡工作在直流模式，当电流绝对值大于 215mA 时，板卡工作在脉冲模式，持续时间为 10mS。直流模式和脉冲模式的分界阈值点由电阻分压得到，为一个近似值；
- (2) 当源工作在上述脉冲模式持续约 10mS 后，电流档位会自动切换到 20mA 电流档位，此时源的输出电流变为之前的 1/25；
- (3) 基于(2)，如果需要再次将源设置成 500mA 电流档位，则需要先将源设置成其他电流档位（除 500mA 档位以外的其他电流档位），再设置成 500mA 电流档位。

relayStatus 设置输出继电器如何动作，分为三种情况(FH, SH, FL, SL):

ACM_RELAY_ON: 打开输出继电器；

ACM_RELAY_OFF: 关闭输出继电器；

ACM_RELAY_HOLD: 输出继电器保持上一个状态。

risingTime double 型变量，设置积分变化时间。单位为 mS，分辨率为 0.1mS。

该参数不写，则默认为 0.1mS；

不同电流档位的最快建立时间不同，如下表所示，当设置值小于该档位最快建立时间时，则按照最快建立时间执行。

• ACM

Set (VIMode *viMode*, double *setValue*, ACM_VRNG *vRange*, ACM_IRNG *iRange*, ACM_OUT_RELAY *relayStatus* = ACM_RELAY_HOLD, double *risingTime* = 0.1);

Range	Fastest Setting Time (ms)
500mA	0.1
200mA	
20mA	
2mA	
200uA	
20uA	0.25
5uA	1.1

样例1: 设置 acm0 电压输出 5V，电流量程选择 ACM_200MA，risingTime 设置为1mS。

```
acm0.Set(FV,0,ACM_N2P18V,ACM_200MA,ACM_RELAY_ON);
delay_ms(1);
acm0.Set(FV,5,ACM_N2P18V,ACM_200MA,ACM_RELAY_ON, 1);
```

• ACM

SetSyn (VIMode *viMode*, const double**setValue*, UINT *siteSize*, ACM_VRNG *vRange*, ACM_IRNG *iRange*, ACM_OUT_RELAY *relayStatus* = ACM_RELAY_HOLD, double *risingTime* = 0.1);

Remarks

对不同的工位设定不同的电压值或电流值，且该操作对不同工位完全同步。

Parameters

viMode ACM 工作模式，有两种模式：

FV: 选择恒压模式

FI: 选择恒流模式

setValue 存放每个工位设定的恒压值或者恒流值的数组

选择恒压模式时，数组中的元素为恒压值，单位为 V。最大范围为-2~18。

选择恒流模式时，数组中的元素为恒流值，单位为 A。最大范围为-0.5~0.5。

siteSize UINT 型变量，设置执行同步操作的工位数。

若该数值为 4，则执行同步操作的工位为 SITE1、SITE2、SITE3、SITE4。

注意：

(1) *setValue* 数组长度必须大于 *siteSize* 的值，否则可能会发生不可预期的错误。

(2) 若 *siteSize* 值小于当前有效工位数，则用户未设定的工位状态不做处理，即保持其原来的状态；

(3) 若 *siteSize* 值大于当前有效工位数，则按当前有效工位数来处理各工位的状态。

• ACM

SetSyn (VIMode *viMode*, const double**setValue*, UINT *siteSize*, ACM_VRNG *vRange*, ACM_IRNG *iRange*, ACM_OUT_RELAY *relayStatus* = ACM_RELAY_HOLD, double *risingTime* = 0.1);

Parameters

vRange ACM 电压量程:
ACM_N2P18V

iRange ACM 电流量程:

ACM_500MA, ACM_200MA, ACM_20MA,
ACM_2MA, ACM_200UA, ACM_20UA, ACM_5UA

ACM_2MA, ACM_200UA, ACM_20UA and ACM_5UA, 称为内部电流量程;
ACM_500MA, ACM_200MA and ACM_20MA, 称为外部电流量程。
当电流挡位在内部量程和外部量程之间切换时, 输出继电器会先断开, 然后再闭合。

Note:

- (1) 选择 ACM_500MA 档位时, 当电流绝对值小于 215mA 时, 板卡工作在直流模式, 当电流绝对值大于 215mA 时, 板卡工作在脉冲模式, 持续时间为 10mS。直流模式和脉冲模式的分界阈值点由电阻分压得到, 为一个近似值;
- (2) 当源工作在上述脉冲模式持续约10mS后, 电流档位会自动切换到 20mA 电流档位, 此时源的输出电流变为之前的 1/25;
- (3) 基于(2), 如果需要再次将源设置成 500mA 电流档位, 则需要先将源设置成其他电流档位 (除 500mA 档位以外的其他电流档位), 再设置成500mA 电流档位。

```
double setValue[2] = {3, 5};
int siteSize = 2;
acm0.SetSyn(FV, setValue, siteSize, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
```

• ACM

SetSyn (VIMode *viMode*, const double**setValue*, UINT *siteSize*, ACM_VRNG *vRange*, ACM_IRNG *iRange*, ACM_OUT_RELAY *relayStatus* = ACM_RELAY_HOLD, double *risingTime* = 0.1);

relayStatus 设置输出继电器如何动作，分为三种情况(FH, SH, FL, SL):

ACM_RELAY_ON: 打开输出继电器;

ACM_RELAY_OFF: 关闭输出继电器;

ACM_RELAY_HOLD: 输出继电器保持上一个状态。

risingTime double 型变量，设置积分变化时间。单位为 mS，分辨率为 0.1mS。

该参数不写，则默认为 0.1mS;

不同电流档位的最快建立时间不同，如下表所示，当设置值小于该档位最快建立时间时，则按照最快建立时间执行。

Range	Fastest Setting Time (ms)
500mA	0.1
200mA	
20mA	
2mA	
200uA	
20uA	0.25
5uA	1.1

• ACM

SetClamp(double *percent_PFS*, double *percent_NFS*);

Remarks

设置 ACM 的正负箝位值。

注意:

(1) SetClamp 函数与模式相关, 切换 FV、FI 模式时会清除箝位设置, 箝位值会自动还原成满箝位值。未切换 FV 和 FI 状态时, 箝位设置会保持不变。

(2) 如果设置的箝位值超出箝位上下限时, 按照上下限执行, 且软件报 **Alarm**。

Parameters

percent_PFS double型变量, 输出正箝位值, 此处指占正箝位满量程的百分比, 单位%。

范围: 箝压 10%~105%

箝流 10%~112%

percent_NFS double型变量, 输出负箝位值, 此处指占负箝位满量程的百分比, 单位%。

范围: 箝压 10%~105%

箝流 10%~112%

• ACM

MeasureVI(ACM_MEASTYPE *measType*, UINT *sampleTimes*, double *samplePeriod*,
FLOATMEASMODE *measMode* = MEAS_NORMAL, UINT *T1* = 0, UINT *T2* = 0, UINT *T3* =
0, UINT *T4* = 0);

Remarks

设置 ACM 的测量结果类型、测量次数、测量间隔、测量模式以及 AWG 测量模式下的四次测量启动时刻。

Parameters

measType 设置 ACM 测量类型。可选值为：

ACM_MV: 测压模式，回读的测量结果为电压值；

ACM_MI: 测流模式，回读的测量结果为电流值。

sampleTimes 设置 AD 采样次数。范围 1~4096。

注意： 当 $sampleTimes < 1$ 时， $sampleTimes = 1$ ；当 $sampleTimes > 4096$ 时， $sampleTimes = 4096$ 。

samplePeriod 设置每次采样之间的时间间隔，单位 uS。最小采样间隔为 5uS，分辨率 1uS。范围：
5~30000。

注意： 当 $samplePeriod < 5$ 时， $samplePeriod = 5$ ；当 $samplePeriod > 30000$ 时， $samplePeriod = 30000$ 。

measMode 测量模式选择，可设置为：

MEAS_NORMAL: 普通测量模式。调用函数后完成测量。

MEAS_AWG: AWG 测量模式。只做设置，运行 AWG 时启动。

该参数默认值为 MEAS_NORMAL 模式

• ACM

MeasureVI(ACM_MEASTYPE *measType*, UINT *sampleTimes*, double *samplePeriod*,
FLOATMEASMODE *measMode* = MEAS_NORMAL, UINT *T1* = 0, UINT *T2* = 0, UINT *T3* =
0, UINT *T4* = 0);

T1、*T2*、*T3*、*T4*

单位 uS。分辨率 10uS，范围：0 ~ (*T1*+*T2*+*T3*+*T4*) ~ 600000。

AWG 测量模式下，设置测量启动时刻。

一次 AWG 中，可以启动四次测量。四个参数均可以不设置，默认启动一次，启动时刻为 0。对于这种测量模式，建议跟同步启动测量配合使用。

注意：

T1、*T2*、*T3*、*T4* 为相对时间。即 *T2* 的启动时间为相对 *T1* 的启动时间，*T3* 的启动时间为相对 *T2* 的启动时间，*T4* 的启动时间为相对 *T3* 的启动时间。

• ACM

GetMeasResult(UINT *siteCount*, int *sampleNumber* = AVERAGE_RESULT, BYTE *triggerNum* = 0);

Remarks

获取 MeasureVI()函数的测量结果。

注意: (1) ACM 每次测量时, 只能单独测量电压或者电流, 因此用户调用此函数的回读结果与 MeasureVI 函数中选择的测量结果类型相关。

(2) 当参数 *sampleNumber* 选择 TRIG_RESULT 时, 如果没有发生触发, 则函数返回值为-3。

Parameters

siteCount UINT 型变量, 选择测量数据工位。0 代表 Site1, 1 代表 Site2.....

sampleNumber INT 型变量, 选择测量结果的数据类型:

AVERAGE_RESULT: 平均值

MAX_RESULT: 最大值

MIN_RESULT: 最小值

TRIG_RESULT: 触发值。AWG 同步测量时, 跳变触发的位置。即 MeasureVI 中第几个采样点发生跳变, 最多支持两次触发。

An int variable 'N': 返回的是第 N+1 个采样点的测量值。

假设 MeasureVI 的采样次数为 sam, 则 $0 \leq N \leq \text{sam}-1$ 。

该参数默认值为 AVERAGE_RESULT。

triggerNum 触发次数设置。

仅当 *sampleNumber* 选择 TRIG_RESULT 时, 该参数有效, 最多支持两次触发。

该参数默认值为 0, 表示回读第一次发生触发的位置。

- ACM

例 1：返回 acm0 电流测量结果平均值。

```
// acm0 MI, sampleTimes=20, samplePeriod=10us
acm0.MeasureVI(ACM_MI, 20, 10);
for(i=0; i<SITENUM; i++)
{
    // the parameter of sampleNumber is default value AVERAGE_RESULT, so the result is average value of current
    adresult[i] = acm0.GetMeasResult(i);
}
```

例 2：返回 acm0 电压测量结果第 15 个点的值。

```
// acm0 MV, sampleTimes=20, samplePeriod=10us
acm0.MeasureVI(ACM_MV, 20, 10);
for(i=0; i<SITENUM; i++)
{
    // return the sample point of 15th of its MV result
    adresult[i] = acm0.GetMeasResult(i, 14);
}
```

• ACM

例 3：AWG 同步测量时，返回 acm0 两次跳变触发的位置。

```
// the trigger value is 2V, falling edge
acm0.SetMeasVTrig(2, TRIG_FALLING);
// acm1 MV, sampleTime=50, samplePeriod=20uS, measure mode is MEAS_AWG
acm1.MeasureVI(ACM_MV, 50, 20, MEAS_AWG);
// the acm0's parameters of MeasureVI function are same with acm1
acm0.MeasureVI(ACM_MV, 50, 20, MEAS_AWG);
// acm1 AWG enable
STSEnableAWG(&acm1);
// acm0 and acm1 measure enable
STSEnableMeas(&acm1, &acm0);
STSAWGRun();
for(i=0; i<SITENUM; i++)
{
    // Get the voltage position of first trigger
    Trig_Point0[i] =acm0.GetMeasResult(0, TRIG_RESULT, 0);
    // Get the voltage position of second trigger
    Trig_Point1[i] =acm0.GetMeasResult(0, TRIG_RESULT, 1);
}
```

• ACM

BlockRead(UINT *siteCount*, UINT *start*, UINT *size*, double* *buffer*)

Remarks

此函数读取一个数据块的测量结果，数据块的起始地址和大小由用户定义。

Note:

通过该函数回读的测量结果与 **MeasureVI** 函数中选择的测量结果类型相关，如果在测量函数中选择的是 **ACM_MV** 模式，则此处回读的为电压的测量结果，反之为电流的测量结果。

Parameters

siteCount UINT 型变量，选择测量数据工位。0 代表 Site1，1 代表 Site2.....

start UINT 型变量，数据块的起始地址，非负整数。

参数范围：0~（**MeasureVI** 函数中设置的采样次数-1）

size UINT 型变量，数据块的大小，正整数。

参数范围：1~（**MeasureVI** 函数中设置的采样次数-start）

buffer 存放数据结果的数组，由用户定义，

如：double result[1000]={0.0} 或 double result[SITENUM] [1000]={0.0};

Note:

(1) *buffer* 数组长度需不小于 *size* 的值，否则可能会发生不可预期的错误；

(2) 建议把该数组定义成全局变量。

- ACM

```
double result[SITENUM][200]={0.0};  
acm0.MeasureVI(ACM_MV, 200, 10); // Measure Voltage  
for(i=0; i<SITENUM; i++)  
{  
    // siteID, start=0; size=200; Get MVRET, saved to result.  
    acm0.BlockRead(i, 0, 200, result[i]);  
}
```

• ACM

Pulse(double *pulseValue*, UINT *pulseTime* = 300, int *measStartTime* = 0, ACM_MEASTYPE *measType* = ACM_MV, UINT *sampleTimes* = 10, double *samplePeriod* = 30)

Remarks

输出幅度、脉宽可调的电压或电流脉冲。

Note:

1) Pulse 函数之前，需先调用 Set 函数来设置量程等信息。Set 函数是用来设置工作模式（恒压/恒流），以及设置电压量程、电流量程；

2) Pulse 脉冲的起始值和终止值均为 Set 语句设置值。

Parameters

pulseValue double 型变量，设置脉冲幅值。

选择 FV 模式时，单位 V。最大范围为-2~18；

选择 FI 模式时，单位 A。最大范围为-0.5~0.5。

pulseTime UINT 型变量，设置脉冲时间。单位 uS。分辨率：10uS。

范围：

300~40000（当源的电流值绝对值小于 215mA 时）；

300~10000（当源的电流值绝对值大于 215mA 时）；

该参数默认为 300uS。

measStartTime int 型变量，设置启动测量时间，单位 uS。范围：-60000~60000。

该参数默认为 0。

measType 设置 ACM 测量类型。可选值为：

ACM_MV: 测压模式，回读的测量结果为电压值；

ACM_MI: 测流模式，回读的测量结果为电流值。

该参数默认为 ACM_MV。

• ACM

Pulse(double *pulseValue*, UINT *pulseTime* = 300, int *measStartTime* = 0, ACM_MEASTYPE *measType* = ACM_MV, UINT *sampleTimes* = 10, double *samplePeriod* = 30)

Parameters

sampleTimes UINT 型变量，设置采样次数。范围 1~4096。
该参数默认为 10。

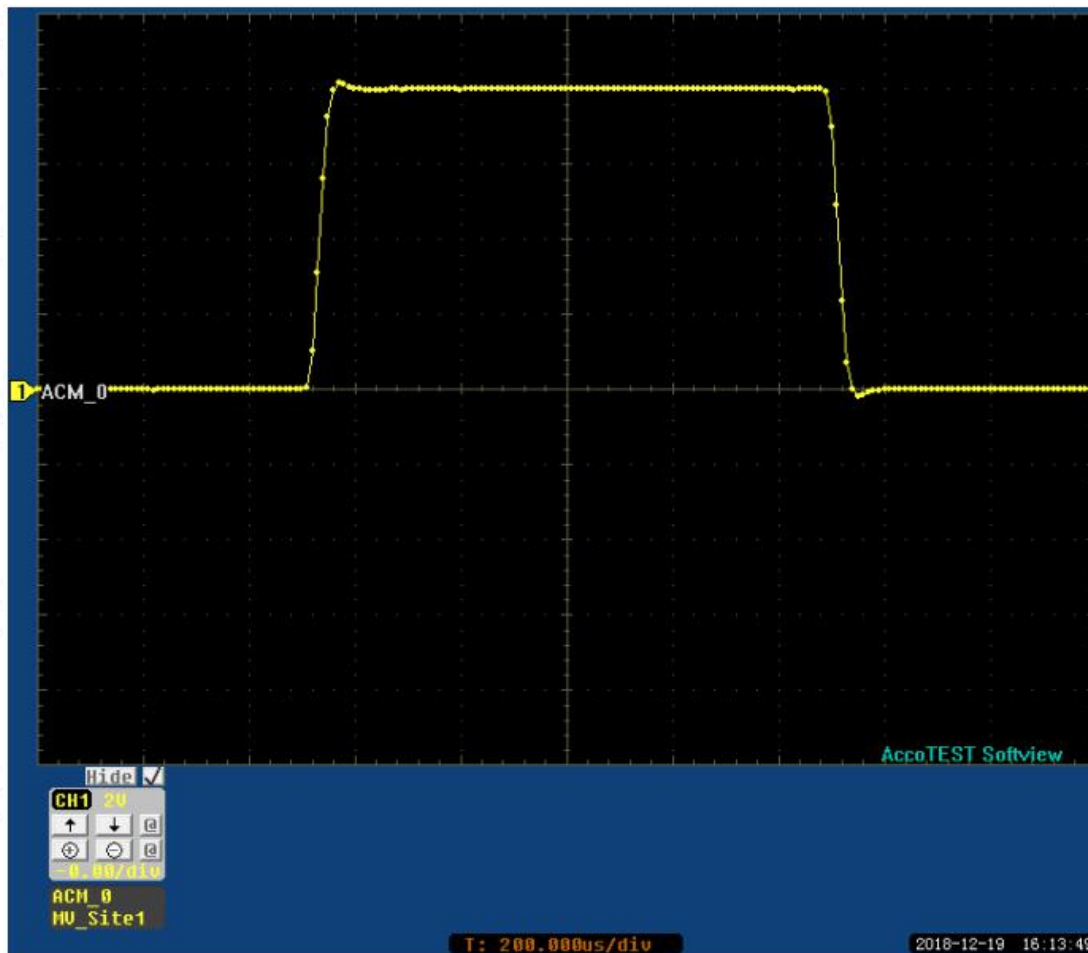
samplePeriod UINT 型变量，设置采样间隔时间，单位 uS，范围：5~30000，分辨率 1uS。
该参数默认为 30uS。

例 1：设置 acm0 起始状态 FV 为 0。设置 acm0 输出 1mS 的 8V 脉冲；测压，采样次数 200 次，采样间隔时间 10uS，脉冲生成前 500uS 开始启动测量。

```
acm0.Set(FV, 0, ACM_N2P18V, ACM_20MA, ACM_RELAY_ON);  
acm0.Pulse (8, 1000, -500, ACM_MV, 200, 10)
```

- ACM

使用 SoftView 软件抓取的波形如下：



- **ACM-AWG**

AwgLoader()

AwgSelect()

AwgRun()

AwgStop()

AwgClear()

SetMeasVTrig()

SetMeasITrig()

• ACM-AWG

AwgLoader (char* *awgName*, VIMode *viMode*, ACM_VRNG *vRange*, ACM_IRNG *iRange*, double* *awgData*, UINT *awgSize*);

Remarks

导入 AWG 波形数据。设置导入的 AWG 波形的名称、导入数据的长度及档位信息。

Note:

- (1) 档位信息不涉及档位的切换，仅用于波形数据换算；
- (2) 同一路 ACM 通道可以导入多个 AWG 波形，但不同 AWG 波形的命名必须不同。对于相同名称的 AWG 波形，只写入第一次调用 AwgLoader 函数中设置的波形；
- (3) 启动 AwgLoader 函数会把本次的 AWG 波形写入 RAM 中，因此执行该语句会占用测试时间。一次性写入 4096 个点的数组，执行 AwgLoader 函数需要 5mS 左右的时间，写入时间与写入点数成正比；
- (4) 同一个测试程序中，同一路 ACM 通道每次导入的 AWG 波形总长度必须小于等于 4096，超出 4096 以后的数据不会写入 RAM；
- (5) 同一个测试程序中，建议同一路 ACM 通道导入的多个 AWG 波形总长度小于 4096。此时，同一个 AWG 波形数据不会被重复导入，仅第一次执行 AwgLoader 函数时占用测试时间；
- (6) 同一个测试程序中，如果同一路 ACM 通道导入的多个 AWG 波形总长度大于 4096。此时，每次会重新写入新的 AWG 数据到 RAM 中，增加了测试时间。

• ACM-AWG

Parameters

awgName char 型变量, AWG 波形名称。命名不区分大小写。

这个命名被应用于AwgSelect()。

viMode ACM 工作模式, 有两种模式:

FV: 选择恒压模式

FI: 选择恒流模式

vRange ACM 电压量程 **ACM_N2P18V**

iRange ACM 电流量程:

ACM_500MA, ACM_200MA, ACM_20MA, ACM_2MA

ACM_200UA, ACM_20UA, ACM_5UA

awgData double 型指针, 存储 AWG 波形数据。

awgSize UINT 型变量, 设置 AWG 波形数据的大小。该参数范围为 1~4096。

例 1: 建立一个波形数据组, acm0 从-1.5V 上升到 3.5V, 步距 0.1V。

```
int sam = 50; //AWG waveform data length
int interval = 200; //AWGdata interval time, unit is uS
double awg_pattern[100]={0.0};
STSAWGCreateRampData(&awg_pattern[0],sam,1,-1.5,3.5);
//pattern array saved in 'awg_pattern', start from -1.5 to 3.5
acm0.AwgLoader("Vst ",FV,ACM_N2P18V,ACM_20MA,awg_pattern,sam);
//pattern named as 'Vst', FV mode
```

• ACM-AWG

AwgSelect (char* *awgName*, UINT *startAddr*, UINT *stopAddr*, int *loopBackAddr*, double *awgInterval* = 10.0);

Remarks

选择将要运行的 AWG 波形，同时设置该 AWG 波形的起始点、结束点、回转点以及 AWG 启动间隔时间。

注意:

- (1) 同一个测试程序中每一路 ACM 通道可以导入多个 AWG 波形，每个 AWG 波形必须有自己的独立名称，通过 AwgSelect 函数来选择将要运行的 AWG 波形。
- (2) AwgSelect 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的 AwgLoader 中的量程档位一致。
- (3) AwgSelect 函数需配合 AWG 同步启动函数使用。执行 AwgSelect 并没有 AWG 输出，只有运行同步启动函数 STSAWGRUN 才会有 AWG 波形输出。

Parameters

- awgName* char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。
- startAddr* UINT 型变量，设置 AWG 波形起始点。
数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。
- stopAddr* UINT 型变量，设置 AWG 波形结束点。
数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。
- loopBackAddr* int 型变量，设置 AWG 波形回转点，即 AWG 执行结束后停止的位置。
当该值设定为-1 时，AWG 波形保持在运行停止的位置；
当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。
- awgInterval* double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。
数据范围为10~60000。默认值为 10uS。

• ACM-AWG

例 1：建立两个波形数据组，分别为 AWG_Vst 和 AWG_Vuvlo。仅运行 AWG_Vst 波形。

```
double interval = 200; // interval time for AWG pattern, unit is uS
int sam1 = 1000; //data size for AWG pattern
int sam2 = 500; // data size for AWG pattern
double awg_pattern1[1000]={0.0}; //define AWG_Vst data array
double awg_pattern2[500]={0.0}; //define AWG_Vuvlo data array
//set waveform data array for AWG_Vst
STSAWGCreateRampData(&awg_pattern1[0],sam1,1,-1.5,3.5);
//set AWG_Vuvlo waveform data array
STSAWGCreateRampData(&awg_pattern2[0],sam2,1,-0.5,-1.5);
// pattern named as 'AWG_Vst', FV mode
acm0.AwgLoader("AWG_Vst",FV,ACM_N2P18V,ACM_200MA,awg_pattern1,sam1);
// pattern named as 'AWG_Vuvlo', FV mode
acm0.AwgLoader("AWG_Vuvlo",FV,ACM_N2P18V,ACM_200MA,awg_pattern2,sam2);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'AWG_Vst' of its 'AwgLoader()'
acm0.Set(FV, -1.5,ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
//Select pattern of 'AWG_Vst', start step=0, stop=sam-1; loop=0; interval time=200uS
acm0.AwgSelect("AWG_Vst", 0, sam1-1, 0, interval);
.....
STSAWGRun(); //AWG synchronous running started for pattern of 'AWG_Vst'
```

• ACM-AWG

AwgRun (char* *awgName*, UINT *startAddr*, UINT *stopAddr*, int *loopBackAddr*,
double *awgInterval* = 10.0, BOOL *runMode* = AWG_SINGLE, UINT *delayTime_ms* = 0);

Remarks

某一路 ACM 通道 AWG 单独启动函数。

注意:

- (1) AwgRun 之前需要设置 Set 语句，该 Set 语句的量程档位必须与所选择的AwgLoader 中的量程档位一致。
- (2) 此函数设置了波形的起始点、结束点、回转点以及时间间隔，不需要配合AwgSelect 函数使用。
- (3) 使用该函数不能与其它源同步测量，需要同步测量时，请使用 AwgSelect 配合同步函数 STSAWGRun。

• ACM-AWG

Parameters

awgName char 型变量，AWG 波形名称。应与之前通过 AwgLoader 函数导入的 AWG 波形名称保持一致。

startAddr UINT 型变量，设置 AWG 波形起始点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

stopAddr UINT 型变量，设置 AWG 波形结束点。

数据范围 0~sam-1。其中 sam 为 AWG 波形的点数。

loopBackAddr int 型变量，设置 AWG 波形回转点。

当该值设定为-1 时，AWG 波形保持在运行停止的位置；

当该值在 0~sam-1 时，AWG 波形运行结束后，会保持在设定点的位置。

awgInterval double 型变量，设置 AWG 数据间隔时间，单位 uS。分辨率为 1uS。

数据范围为10~60000。该参数默认 10uS。

runMode 运行模式选择，可以设定为：

AWG_SINGLE 运行一次 AWG 波形。

AWG_LOOP 循环运行 AWG 波形，使用 AwgStop()函数停止。

默认值为 AWG_SINGLE。

delayTime_ms UINT 型变量，单位 mS。范围：0~60000。设置延时时间。

注意：

1)AWG 输出 delayTime_ms 之后，系统允许 AwgRun 之后的操作开始运行。

2) delayTime_ms 不填的时候，delayTime_ms 默认为 AWG 扫描时间，只有在 AWG 扫描完毕之后，系统才允许 AwgRun 之后的操作开始运行。建议不填该参数！

• ACM-AWG

例 1: acm0 连续输出 120Hz 正弦波，峰峰值 0.66V，直流偏置为 3.3V。

```
int sam = 830; //size of AWG pattern
double interval = 10; //interval time of AWG pattern, unit is uS
double awg_pattern[1000]={0.0};
// set waveform data array for Sin_AWG
STSAWGCreateSineData(&awg_pattern[0],sam,1,0.66,3.3,0);
// pattern named as 'Sin_AWG, FV mode
acm0.AwgLoader("Sin_AWG",FV, ACM_N2P18V, ACM_500MA,awg_pattern,sam);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its 'AwgLoader()'
acm0.Set(FV, 0, ACM_N2P18V, ACM_500MA, ACM_RELAY_ON);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS,loop mode
acm0.AwgRun("Sin_AWG ",0,(sam-1),0, interval,AWG_LOOP);
```

• ACM-AWG

AwgStop(void);

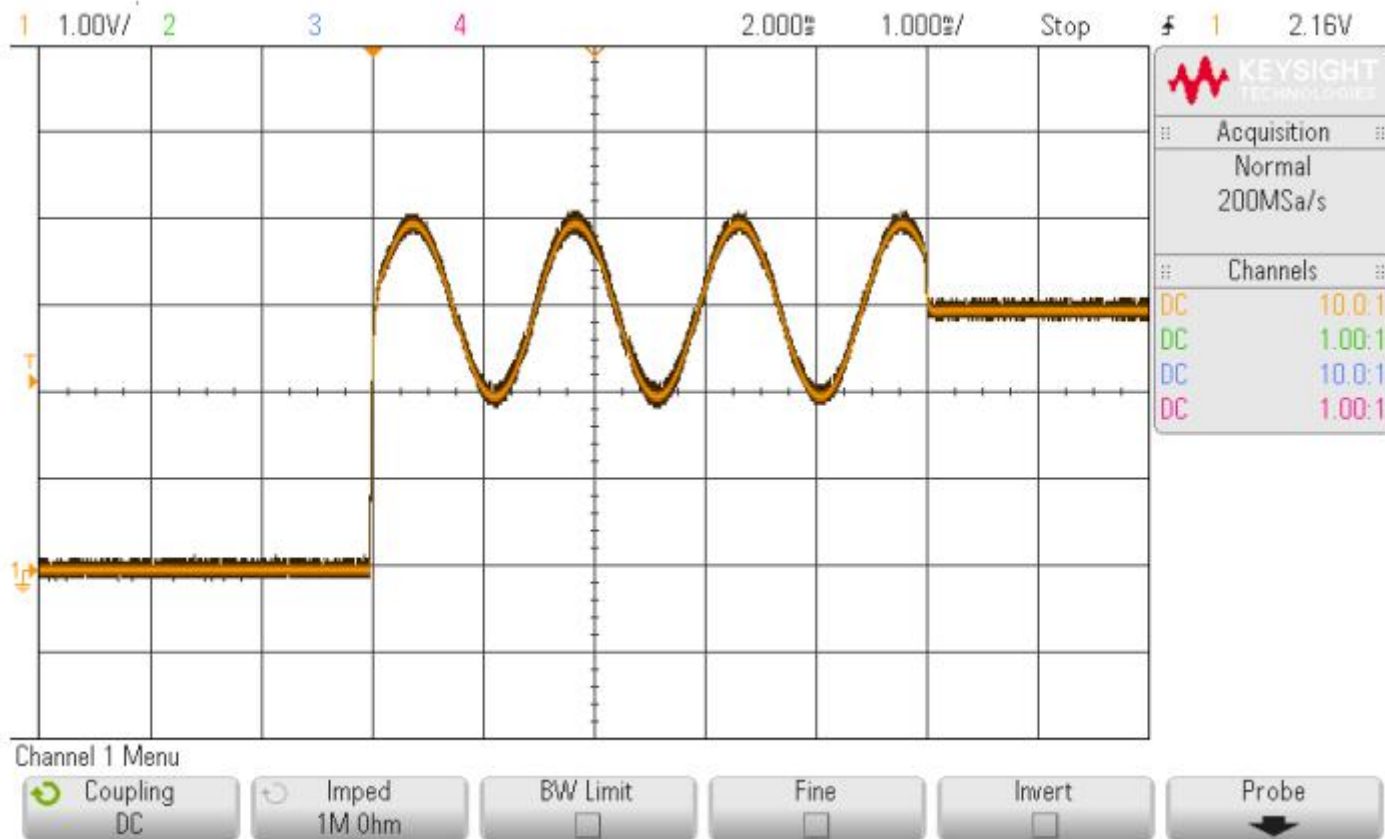
Remarks

停止某一路 ACM 通道的 AWG 循环运行。

例 1: acm0 生成一个周期为 1.5mS 的正弦波，循环运行该正弦波，5mS 之后停止运行。

```
int sam = 150; //data size for AWG pattern
double interval = 10; // interval time for AWG pattern, unit is uS
double awg_pattern[1000]={0.0};
STSAWGCreateSineData(&awg_pattern[0],sam,1,2,3,0);
//set a sinewave data array, Vpp=2, DC Offset=3
acm0.AwgLoader("Sin_AWG", FV, ACM_N2P18V, ACM_500MA, awg_pattern, sam);
// pattern named as 'Sin_AWG'
acm0.Set(FV, 0, ACM_N2P18V, ACM_500MA, ACM_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Sin_AWG' of its AwgLoader()
acm0.AwgRun("Sin_AWG", 0, (sam-1), 0, interval, AWG_LOOP);
//Select pattern of 'Sin_AWG', start step=0, stop=sam-1; loop=0; interval time=10uS, loop mode
delay_ms(5);
acm0.AwgStop(); //stop running AWG waveform of acm0
```

该正弦波周期为 1.5mS，延时 5mS 后，停止 awg 波形。
下图为示波器抓取的 actual 波形。



- ACM-AWG

```
AwgClear(void);
```

Remarks

一般用于 AwgLoader 函数之前，清除该 ACM 通道的 AWG 数据。

例 1：Load 波形之前清除 acm0 通道的 AWG 数据。

```
acm0.AwgClear(); //clear AWG data of acm0  
acm0.AwgLoader("P1", FV, ACM_N2P18V, ACM_200MA, awg_pattern1, sam)
```

• ACM-AWG

```
SetMeasVTrig(double vTrig, TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Remarks

电压测量触发设置。

注意:

SetMeasureVTrig 函数与 MeasureVI 函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Parameters

vTrig double 型变量，设置触发电压。单位 V。

Note:

该参数的值与其对应通道所调用的 Set() 函数的电压档位相关。

trigMode 触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数默认为 TRIG_FALLING 下降沿触发。

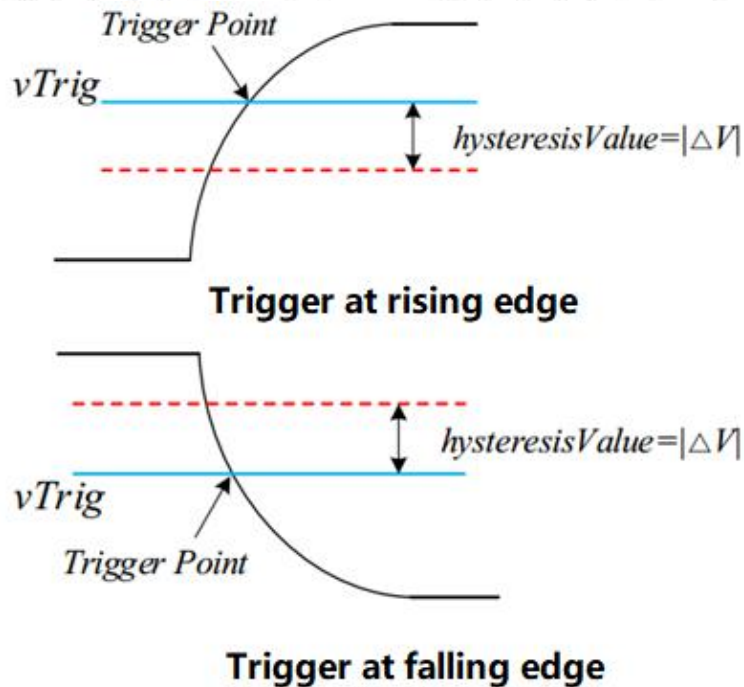
hysteresisValue double 型变量，设置触发电压的滞回值。单位 V。

在默认条件下等于 0。

• ACM-AWG

注意:

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值;
- 2) 在上升沿 (TRIG_RISING) 触发的情况下, 当判定到测量结果先小于 ($vTrig - hysteresisValue$) 值之后又大于 $vTrig$ 值时, 此时在 $vTrig$ 处判定为触发;
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下, 当判定到测量结果先大于 ($vTrig + hysteresisValue$) 值之后又小于 $vTrig$ 值时, 此时在 $vTrig$ 处判定为触发。



• ACM-AWG

```
SetMeasITrig(double iTrig, TRIG_MODE trigMode = TRIG_FALLING,  
double hysteresisValue = 0.0);
```

Remarks

电流测量触发设置。

注意:

SetMeasITrig 函数与 MeasureVI 函数运行时，对应的电压电流档位状态必须一致。二者之间不允许插入改变电压电流量程档位的函数。

Parameters

iTrig double 型变量，设置触发电流。单位 A。

注意:

该参数的值与其对应通道所调用的 Set() 函数的电流档位相关。

trigMode 触发模式设置，可设置为：

TRIG_FALLING 下降沿触发

TRIG_RISING 上升沿触发

该参数默认为 TRIG_FALLING 下降沿触发。

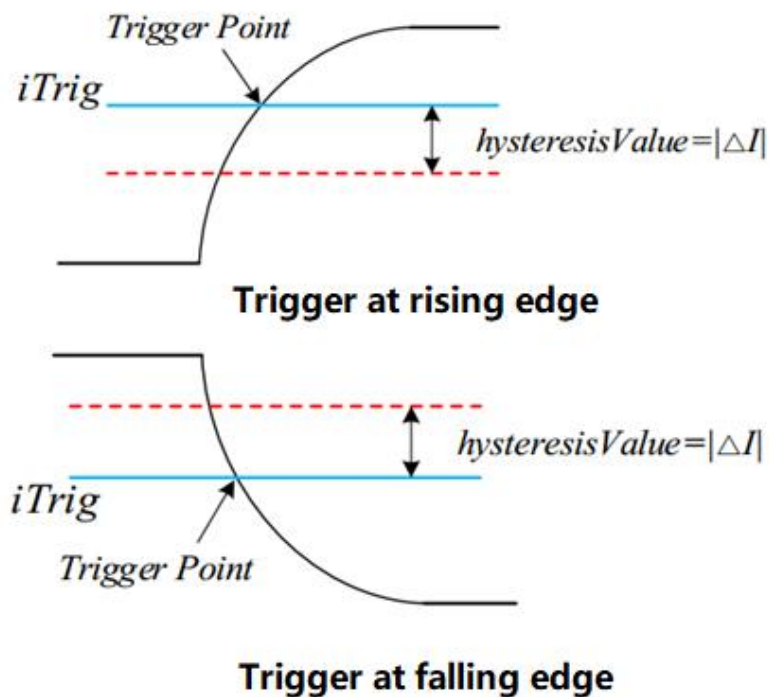
hysteresisValue double 型变量，设置触发电流的滞回值。单位 A。

在默认条件下等于 0。

• ACM-AWG

注意:

- 1) 滞回参数 *hysteresisValue* 仅可设置为非负值;
- 2) 在上升沿 (TRIG_RISING) 触发的情况下, 当判定到测量结果先小于($iTrig - hysteresisValue$)值之后又大于 $iTrig$ 值时, 此时在 $iTrig$ 处判定为触发;
- 3) 在下降沿 (TRIG_FALLING) 触发的情况下, 当判定到测量结果先大于($iTrig + hysteresisValue$)值之后又小于 $iTrig$ 值时, 此时在 $iTrig$ 处判定为触发。



• ACM-AWG

acm0: 建立一个波形数据组, acm0从 10.5V 降到 8V, 步距为 0.1V.

acm1: 测试 acm1 端的电压小于 2V 时, acm0 跳变点的电压值。

```
acm1.Set(FI, 0, ACM_N2P18V, ACM_20MA, ACM_RELAY_ON);
double Trig = 2; //triggering voltage
double Trig_Point[4]={0.0};
double result[4]={0.0};
int sam = 25;
double interval = 200;
double awg_pattern[100]={0.0};
STSAWGCreateRampData(&awg_pattern[0],sam,1,10.5,8);
//Set a ramp wave data array, data size is 25, period is 1, from 10.5 to 8.
acm0.AwgLoader("Uvlo",FV,ACM_N2P18V,ACM_200MA,awg_pattern,sam);
//import the awg_pattern array into acm0 channel, named it as 'Uvlo'. Set to FV mode
acm0.Set(FV, 10.5, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
// Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Uvlo' of its 'AwgLoader()'
acm0.AwgSelect("Uvlo", 0, sam-1, sam-1, interval);
//Select pattern of 'Uvlo', start step=0, stop=sam-1; loop= sam-1; interval time=200uS
acm1.SetMeasVTrig(Trig, TRIG_FALLING);
//trigger value is 2V, falling edge
acm0.MeasureVI(ACM_MV, sam,interval, MEAS_AWG);
acm1.MeasureVI(ACM_MV, sam,interval,MEAS_AWG);
STSEnableAWG(&acm0); //enable AWG pattern for acm0
STSEnableMeas(&acm0, &acm1); //enable measurement for acm0 and acm1
STSAWGRun(); //Enable AWG and measurement synchronously
```

当调用 **SetMeasVTrig/SetMeasITrig** 以及 **MeasureVI**时, 电流量程必须保持一致, 禁止在两个指令之间, 更改电流量程。

• ACM-AWG 样例

```
DUT_API int test_Example2(short funcindex, LPCTSTR funclabel)
{
    //{AFX_STG_PARAM_PROTOTYPES
    CParam *Vcspre = StsGetParam(funcindex, "Vcspre");
    //}AFX_STG_PARAM_PROTOTYPES
    double Trig = 2;// trigger voltage
    double Trig_Point[STS_SITE_NUM] = { 0.0 };
    double result[STS_SITE_NUM] = { 0.0 };
    int sam = 100;// data size of AWG pattern
    int interval = 100;// interval time of AWG pattern, unit is uS
    double awg_pattern[100] = { 0.0 };
    acm1.Set(FI, 0, ACM_N2P18V, ACM_2MA, ACM_RELAY_ON);// OUT Measure
    delay_ms(1);
    // Set an ramp data array, the start address starts from 0, data size is 100.
    STSAWGCreateRampData(&awg_pattern[0], sam, 1, 0.35, 0.45);
    // import the awg_pattern array into acm0 channel, named it as 'Vcspre_AWG'. Set to FV mode.
    acm0.AwgLoader("Vcspre_AWG", FV, ACM_N2P18V, ACM_200MA, awg_pattern, sam);
    // Call 'Set()' to make sure that the mode and ranges are exactly the same with 'Vcspre_AWG' of its 'AwgLoader()'
    acm0.Set(FV, 0.35, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
    delay_ms(5);
    // Select pattern of 'Vcspre_AWG', start step=0, stop=sam-1; loop= sam-1; interval time=100us
    acm0.AwgSelect("Vcspre_AWG", 0, sam - 1, sam - 1, interval);
    acm1.SetMeasVTrig(Trig, TRIG_FALLING);// trigger value is 2V, Falling edge
    acm0.MeasureVI(ACM_MV, sam, interval, MEAS_AWG);
    acm1.MeasureVI(ACM_MV, sam, interval, MEAS_AWG);
    STSEnableAWG(&acm0);// Enable AWG pattern for acm0
    STSEnableMeas(&acm0, &acm1);// Enable measurement for acm0 and acm1
    STSAWGRun();// Enable AWG and measurement synchronously

    //To be continued.....
}
```

• ACM-AWG 样例

//Continued on previous page

```
for (int i = 0; i < STS_SITE_NUM; i++)
{
    Trig_Point[i] = acm1.GetMeasResult(i, TRIG_RESULT); // Get the position of trigger point for acm1
    result[i] = acm0.GetMeasResult(i, Trig_Point[i]); // Read the voltage value of acm0 on trigger position
    Vcspre->SetTestResult(i, 0, result[i]);
}

acm1.Set(FI, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_ON);
delay_ms(1);
acm1.Set(FV, 0, ACM_N2P18V, ACM_200MA, ACM_RELAY_OFF);
delay_ms(1);
acm0.Set(FV, 0, ACM_N2P18V, ACM_2MA, ACM_RELAY_OFF);

return 0;
}
```



Make Testing More Valuable